TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Context Files: File Management and Data Collection Strategies

An HPE Juniper Networking Techpost

Christian Graf

2026-04-21

Contents

1 Context Files: File Management and Data Collection Strategies 2

1.1 Introduction . . . 2
1.2 The Phase Model . . . 3
1.3 Phase 0: Collect . . . 4
1.4 Why a Dedicated Collection Phase? . . . 4
1.5 Phase 0 Example . . . 5
1.6 Master Index Pattern . . . 6
1.7 Why Master Index? . . . 6
1.8 Master Index Structure . . . 6
1.9 Using the Master Index . . . 9
1.10 Token Savings . . . 9
1.11 File Naming Conventions . . . 11
1.12 Common approaches - and why they fall short . . . 11
1.13 Why Unix epoch wins for cache files . . . 12
1.14 Directory structure - phase as a folder, not a filename prefix . . . 12
1.15 Interface Transformations . . . 13
1.16 The Problem . . . 13
1.17 Solution: Pre-compute and Cache . . . 13
1.18 Split Workflows for Clarity and Reliability . . . 14
1.19 Split Into Phases . . . 14
1.20 Key Benefits . . . 15
1.21 Data Sharing Between Phases . . . 15
1.22 Phase 2: Correlate . . . 17
1.23 Phase 3: Trend . . . 18
1.24 Generating a Trending Context File . . . 18
1.25 Summary . . . 19
1.26 Next in the Series . . . 19
1.27 Glossary . . . 20

1 Context Files: File Management and Data Collection Strategies

Christian Graf - 04/22/2026

Fourth part of the Series on the Context-Files and JMCP, covering tokens, data formats, and memory management.

Series Navigation:

- Part 1: Introduction & Basics
- Part 2: Tokens & Memory
- Part 3: Optimization & Alerting
- Part 4: File Management <- You are here
- Part 5: Redis & Performance
- Part 6: Hybrid Architecture

1.1 Introduction

Parts 1–3 covered context file fundamentals, token management, and optimization. Part 1 also introduced the enhanced JMCP server, which persists collected data directly to Redis or files - tool output never reaches the LLM context window. Instead of returning raw CLI data, the enhanced JMCP returns a brief summary (router count, run status, and a Redis key) totalling roughly 200 tokens, regardless of how large the underlying dataset is.

But what if the MCP server does not support direct storage? To understand the problem concretely, the author ran **show isis database extensive** across 74 ISIS-meshed cRPD instances. The raw output is 26.9 MB, measured at 9,952,294 tokens using tiktoken. By any reasonable estimate, injecting that payload directly into LLM context would put severe pressure on, or outright exhaust, most models' context windows.

The test was run under VS Code with GitHub Copilot and Claude Sonnet 4.6. The context window survived, but not because the LLM handled 10 million tokens. VS Code's Copilot chat layer intercepted the oversized tool result and wrote it to a temporary file on disk, providing the model with a file reference rather than raw text. **This is a client-side guardrail specific to VS Code, not a property of the MCP protocol.** A raw API client - Python SDK, curl, a CI pipeline, any client without a similar guardrail - would have received all 26.9 MB inline and likely exhausted the context window immediately.

Takeaway before reading further: If the goal is handling large datasets reliably and portably, the most robust solution is the enhanced JMCP server's artifact mode. It addresses the problem at the source - data stays in Redis, context stays clean, and the behaviour is consistent regardless of client.

This part is about what to do **when that option is not available** - how to use local file saving to protect the context window, keep data across sessions, and structure multi-phase workflows that scale.

Who this part is for, pick your scenario:

- **Enhanced JMCP in artifact mode** (server stores output directly to Redis/files, LLM context never sees raw data): the patterns in this part apply conceptually, but the mechanics are largely handled for you. You can skim for the Phase 0 (the data collection phase) and Master Index design principles, then move on to the next blog in this series.

- **Standard JMCP or any MCP server without built-in storage** (tool output is returned to the client - whether it enters LLM context raw depends on the client layer; see Introduction): this part is essential reading. It explains how to instruct the LLM to save, organize, and retrieve data across sessions to avoid context window exhaustion.
- **Vanilla Junos MCP or other MCP servers without file/Redis tools**: the same guidance applies - the LLM orchestrates all file I/O through available tools or Python, following the patterns described here.
- **Network health trending over time** (detecting recurring failures, regressions, first-occurrence events, and improving or oscillating health across repeated workflow runs): the Phase 3: Trend section is for you. It requires at least two complete Phase 0–2 runs and can be triggered on demand or as an automated post-Phase-2 step. Redis TTL-based expiry of old runs and look-back window configuration are covered in Part 5.

The workflow this part teaches is built around four named phases:

- **Phase 0 (Collect)** - query all routers once, save each command's output to a cache file immediately, write a Master Index;
- **Phase 1 (Analyse)** - parallel sub-tasks, each scoped to one data domain, each loading a targeted subset of the Phase 0 cache and producing its own structured report;
- **Phase 2 (Correlate)** - load all Phase 1 reports, identify cross-domain causal chains per router, write an executive summary;
- **Phase 3 (Trend)** - compare Phase 0 snapshots and Phase 2 summaries across multiple runs over time to detect recurring failures and drift.

These phases represent a workflow architecture, not just a workaround for missing tooling. They apply equally with enhanced JMCP in artifact mode - even when data storage is handled server-side, the benefits of scope isolation, parallel execution, and failure isolation come from how the workflow is structured, not from how data is stored.

Scope: The patterns in this part are designed for pre-defined, repeatable workloads - scenarios where all required commands are known upfront and the workflow is fixed (e.g., a scheduled ISIS health check across 500 routers). They are not well-suited to ad-hoc or exploratory investigations, where the next command depends on what the previous one revealed. For exploratory work, the handover pattern from Part 2 is a better fit.

1.2 The Phase Model

The four phases form a pipeline. Each can run in its own LLM session, with a clean context window and a file-based handoff to the next phase.

Phase	Name	Scope	Oupout
0	Collect	Query all devices in parallel, save every command to a cache file, write the Master Index	results/cache/

Phase	Name	Scope	Oupout
1	Analyse	One sub-task per data domain - each reads a targeted subset of the cache and writes a structured domain report	results/phase1/
2	Correlate	Load all Phase 1 reports, identify cross-domain causal chains per router, write the executive summary	results/phase2/
3	Trend	Compare Phase 0 snapshots and Phase 2 summaries across multiple runs - detect recurring failures, first occurrences, and improving or worsening health over time	results/phase3/

Phase 1 sub-tasks (e.g., 1a: LSDB, 1b: Adjacency & Interface, 1c: Performance) run in parallel - Phase 2 waits for all of them to complete. Phase 3 is optional and requires at least two complete collect-through-correlate runs. Each phase is described in its own section below.

1.3 Phase 0: Collect

When working with large-scale network automation, separating data collection from analysis provides significant benefits. That's why the enhanced JMCP server includes built-in tooling to read, write, and search a Redis memory cache directly. **As noted in the Introduction, this article focuses on what to do when that tooling is not available.**

The **Phase 0 pattern** collects and caches all required data once, then multiple analysis phases can reuse it without repeated queries.

1.4 Why a Dedicated Collection Phase?

Problems with Monolithic Workflows:

- Repeated JMCP queries for the same data across different analysis steps
- Token accumulation from carrying all raw data in a single LLM session
- Difficult to isolate failures - one analysis failure means re-running everything
- Hard to parallelize - analysis steps must run sequentially

Phase 0 Solution:

- **Collect once, analyze many times**
- With the enhanced JMCP server, output is stored directly to Redis or files by the server itself - the LLM context window is never contaminated, even during collection
- With standard JMCP, the tool result is returned to the client - whether it enters LLM context raw depends on the client (VS Code offloads large responses to disk; a raw API client receives the full payload inline); saving to files still pays off across session boundaries by avoiding re-collection
- Subsequent phases load only the subset they need from cache, skipping full re-collection entirely
- Each analysis phase gets a fresh token budget (new session)
- Parallel execution possible after Phase 0 completes
- Failure isolation - re-run only failed phases

Empirically verified (74 cRPD routers): **show isis database extensive** produces ~365 KB per router - 27 MB for 74 routers. A single batch run via enhanced JMCP (artifact/Redis mode) completed in 3.4 seconds: 74/74 routers succeeded, 26.9 MB stored to Redis, and only ~200 tokens reached the LLM context window. Integrity check: 74 LSPs per router, consistent LSDB across all nodes, full sequence-number fields present - VERDICT: PASS.

A canary-rule test (see blog2 of this series) was also run with standard JMCP on the same 74 routers. The canary survived post-run - but for a non-obvious reason: VS Code Copilot chat intercepted the 27 MB tool result and wrote it to a file on disk instead of injecting it as raw text into the model's context. This is a **client-side guardrail specific to VS Code**, not a property of the MCP protocol. A raw API client (Python SDK, curl, any other frontend) would have received all 27 MB inline and exhausted the context window immediately. Enhanced JMCP artifact mode is the most reliably client-agnostic approach: the oversized data never leaves the server regardless of client.

The success of upfront, file-based data collection depends heavily on the LLM and client environment in use. If it is not clear why the enhanced JMCP server eliminates this risk entirely, revisit the Introduction before continuing.

The following is an example Phase 0 context file. It reduces context window pressure by instructing the LLM to save each command's output to disk immediately - so that by the time the next command returns, the previous result has already been offloaded.

1.5 Phase 0 Example

Goal: Collect all ISIS-related data from 74 routers for health analysis

```
## Phase 0: ISIS Data Collection

**Description:** Collect all ISIS-related data and cache for subsequent analysis phases

**Commands to Execute (JMCP batch, parallel):**
1. `show isis database | display json | no-more`
2. `show isis adjacency | display json | no-more`
3. `show isis interface | display json | no-more`
4. `show isis overview | display json | no-more`
5. `show isis statistics | display json | no-more`
6. `show ddos-protection protocols isis | display json | no-more`
7. `show interfaces extensive | display json | no-more`
8. `show system resources | display json | no-more`

**Procedure:**
Execute one command at a time across all routers (parallel per-router, sequential per-command).
```

```
Save each output to file immediately before proceeding to the next command.
This keeps the context window clean: by the time command 2 arrives, command 1's raw output has been saved and can be compressed/forgotten by the LLM.

1. Execute `show isis database` on all routers (parallel) -> extract key fields -> save to `results/cache/isis_database_{epoch_timestamp}.json` -> proceed
2. Execute `show isis adjacency` on all routers (parallel) -> extract key fields -> save to `results/cache/isis_adjacency_{epoch_timestamp}.json` -> proceed
3. Execute `show isis interface` on all routers (parallel) -> extract key fields -> save to `results/cache/isis_interface_{epoch_timestamp}.json` -> proceed
4. Execute `show isis overview` on all routers (parallel) -> extract key fields -> save to `results/cache/isis_overview_{epoch_timestamp}.json` -> proceed
5. Execute `show isis statistics` on all routers (parallel) -> extract key fields -> save to `results/cache/isis_statistics_{epoch_timestamp}.json` -> proceed
6. Execute `show ddos-protection protocols isis` on all routers (parallel) -> extract key fields -> save to `results/cache/ddos_protection_{epoch_timestamp}.json` -> proceed
7. Execute `show interfaces extensive` on all routers (parallel) -> extract key fields -> save to `results/cache/interfaces_extensive_{epoch_timestamp}.json` -> proceed
8. Execute `show system resources` on all routers (parallel) -> extract key fields -> save to `results/cache/system_resources_{epoch_timestamp}.json` -> proceed
9. Generate master index per the Step 9 helper prompt in the Master Index Pattern section
10. Save master index: `results/cache/isis_data_collection_{epoch_timestamp}.json`

> **Why sequential per-command?** All 8 commands fired in a single batch would flood the context window simultaneously with no opportunity to offload. Executing one command at a time lets the LLM save and release each output before the next arrives. Per-router parallelism (all routers in one batch call) is preserved - only the per-command ordering changes.

**Token Impact:** ~200 tokens with enhanced JMCP (empirically measured - server stores directly to Redis, LLM context never sees raw output); with standard JMCP the 27 MB tool result is delivered to the client - whether it reaches the LLM context depends on the client: VS Code Copilot chat offloads large tool responses to disk, while a raw API client receives the full payload inline and will exhaust the context window.

**Next:** End this session. Start new sessions for each analysis phase (token budget reset per phase).
```

1.6 Master Index Pattern

The Master Index is a critical pattern that ensures temporal consistency when comparing data across multiple routers and commands.

1.7 Why Master Index?

Problem: Inconsistent timestamps across collected data

Assume LSDB snapshots collected at different times:

- isis:lsdb:R1:**1733472000** (collected at 10:00 AM)
- isis:lsdb:R2:**1733475600** (collected at 11:00 AM - 1 hour later)
- isis:lsdb:R3:**1733472000** (collected at 10:00 AM)

For LSDB sync checks, all router snapshots must be collected within a narrow time window - ideally in a single parallel batch. The Master Index records the collection timestamp per router, giving analysis phases a reliable reference for detecting and flagging temporal skew.

1.8 Master Index Structure

The LLM generates this file at the end of Phase 0. The structure below shows the target schema - the helper prompt in the next block is what causes the LLM to produce it.

> **Note**: There is no single required format for a Master Index. A plain JSON file, a YAML document, a SQLite database, or even a Redis hash could serve the same purpose. The preamble + JSON hybrid used here is one option that works well in practice: it is human-readable without tooling, parseable without a schema, and the plain-text header makes critical warnings visible the instant a file is opened - even before the JSON is loaded. If your workflow already has an established metadata format, adapt the principles (temporal skew check, expiry field, per-router status) to fit it rather than adopting this layout wholesale.

The file uses a **preamble + JSON hybrid**: critical status information is written as plain text at the top of the file, so any LLM or human loading it immediately sees warnings without having to parse JSON. The JSON block below the preamble contains the structured data for programmatic lookup.

Example output - clean run (no temporal skew):

```
PHASE 0 STATUS : success | 74/74 routers | 0 failures
COLLECTED : 2025-02-21T14:30:00Z (duration: 45.2s)
CACHE EXPIRES : 2025-02-28T14:30:00Z
TEMPORAL CHECK : OK - all routers within 60s window
WARNINGS : none

 -
```

Example output - with temporal skew:

```
PHASE 0 STATUS : partial | 72/74 routers | 2 failures
COLLECTED : 2025-02-21T14:30:00Z (duration: 87.3s)
CACHE EXPIRES : 2025-02-28T14:30:00Z
TEMPORAL CHECK : [!] WARNING - 3 routers collected >60s after earliest
  8700k_crpd12 : collected 73s after earliest
  8700k_crpd31 : collected 91s after earliest
  8700k_crpd67 : collected 118s after earliest
  -> LSDB sync analysis results may be unreliable for these routers.
WARNINGS : 8700k_crpd5 - SSH timeout, 8700k_crpd41 - SSH timeout

 -
```

Followed immediately by the JSON block for structured lookup:

```
{
  "metadata": {
    "collection_timestamp": "2025-02-21T14:30:00Z",
    "collection_duration_seconds": 45.2,
    "cache_expiry": "2025-02-28T14:30:00Z",
    "cache_ttl_days": 7,
    "routers_collected": 74,
    "routers_failed": 0
  },
  "execution_summary": {
    "stage_1_jmcp_query": "success",
    "stage_2_parsing": "success",
    "stage_3_file_storage": "success",
    "stage_4_completion": "success"
  },
  "cache_files": {
    "isis_database": "results/cache/isis_database_1733472000.json",
    "isis_adjacency": "results/cache/isis_adjacency_1733472000.json",
    "isis_interface": "results/cache/isis_interface_1733472000.json",
    "isis_overview": "results/cache/isis_overview_1733472000.json",
    "isis_statistics": "results/cache/isis_statistics_1733472000.json",
    "ddos_protection": "results/cache/ddos_protection_1733472000.json",
    "interfaces_extensive": "results/cache/interfaces_extensive_1733472000.json",
    "system_resources": "results/cache/system_resources_1733472000.json"
  },
  "router_status": {
    "R1": {"status": "success", "collection_timestamp": 1733472000},
    "R2": {"status": "success", "collection_timestamp": 1733472000},
    "R3": {"status": "success", "collection_timestamp": 1733472000}
  },
  "temporal_warnings": []
}
```

The preamble is what an LLM in an analysis phase session reads first - warnings are immediately visible without any parsing. The JSON is what it parses to locate cache file paths.

Note the deliberate format split: top-level timestamps (collection_timestamp, cache_expiry) are ISO 8601 - they appear verbatim in the human-readable preamble. Per-router timestamps in router_status are Unix epoch integers - skew detection (collected Ns after earliest) is then a plain integer subtraction with no parsing required.

A natural question at this point is: how would you write this kind of prompt yourself? You don't have to - ask the LLM. The helper prompt below was generated from something close to a single line:

Please create a snippet which creates a master-index when storing files to ensure temporal consistency and completeness. Use unix epoch timestamps in file names. Add a summary-header in front. When reading data from files, guide me how to use the master-index for consistency checks

The epoch instruction is worth calling out - it is a small addition to the prompt, but it shapes the entire naming scheme. As covered in the File Naming Conventions section, Unix epoch timestamps are lexicographically sortable, timezone-agnostic, and glob-friendly: the LLM can find the latest cache file with a simple ***** pattern and compare expiry values with a plain integer check. Leaving that out and letting the LLM pick a format freely often results in ISO 8601 strings or human-readable date prefixes - both technically correct, but harder to work with programmatically.

The helper prompt below achieves exactly the same result with a much more explicit specification - every field, every threshold, every condition spelled out. It even covers how to get help when retrieving data from files. This is not the first time in this series where two prompts of very different size and complexity produce comparable output: today's LLMs have a reasonably strong grasp of data integrity and good practices, and whether given a one-liner or a detailed schema, they tend to fill in the gaps sensibly.

The rule of thumb is to start with simple prompts, as shown above - rather than jumping straight to the detailed version below. Grow and extend in phases, always trying with the help of the LLM to keep the context-file crisp and sharp.

Helper prompt - add this as the final step in your Phase 0 context file:

```
**Step 9: Generate Master Index**

Once all commands have completed and their outputs are saved, generate the master index file
`results/cache/isis_data_collection_{epoch_timestamp}.json` using the following structure:

**Part 1 - plain-text preamble (written first, before the JSON):**
- Line 1: `PHASE 0 STATUS : <success|partial|failed> | <N>/74 routers | <F> failures`
- Line 2: `COLLECTED : <ISO 8601 timestamp> (duration: <seconds>s)`
- Line 3: `CACHE EXPIRES : <ISO 8601, 7 days from now>`
- Line 4: `TEMPORAL CHECK : OK - all routers within 60s window`
  OR if any router's collection timestamp differs from the earliest by >60s:
  `TEMPORAL CHECK : [!] WARNING - <N> routers collected >60s after earliest`
  followed by one indented line per affected router:
  ` <router> : collected <N>s after earliest`
  and a final indented note:
  ` -> LSDB sync analysis results may be unreliable for these routers.`
- Line 5: `WARNINGS : none`
  OR a comma-separated list of `<router> - <error>` for any failed routers
- Then a blank line and ` - `

**Part 2 - JSON block (written after the preamble):**
- `metadata`: ISO 8601 collection timestamp, duration in seconds, cache expiry,
  TTL days (7), routers_collected, routers_failed
- `execution_summary`: success/failure for stages: jmcp_query, parsing,
  file_storage, completion
- `cache_files`: exact file path written for each of the 8 commands
- `router_status`: for every router, its status ("success" or error string)
  and its individual collection timestamp as Unix epoch
- `temporal_warnings`: list of strings `"<router>: collected <N>s after earliest"`
```

```
  for any router exceeding the 60s threshold (empty list if none)

Save the file. Confirm the path, router count, and whether any temporal warnings
were raised.
```

At this stage in Phase 0, data has been collected, saved into files, and enriched with consistency checks. Now it's time to read that data from file for analysis.

1.9 Using the Master Index

Three things to notice in the snippet below:

- 1. **Preamble before JSON** - the LLM reads the plain-text header first. Temporal warnings and failed routers are visible without any parsing.
- 2. **Two-gate validation** - preamble status check (PHASE 0 STATUS) + JSON expiry/completion check before touching any data file.
- 3. **Built-in fallback** - if the cache is absent, expired, or failed, the phase degrades gracefully to live queries rather than crashing silently.

Analysis Phase Context File:

```
# Phase 1a: LSDB Analysis

**Data Source:**
- **Primary:** Master cache index `results/cache/isis_data_collection_{latest}.json`
  1. Read preamble first: check `PHASE 0 STATUS` (must be `success` or `partial`),
     `CACHE EXPIRES` (must be > now - abort if expired),
     `TEMPORAL CHECK` (surface any skew warnings before proceeding),
     `WARNINGS` (note any failed routers)
  2. Parse JSON: confirm `metadata.cache_expiry` > now and
     `execution_summary.stage_4_completion` == "success"
  3. Use `cache_files.isis_database` for LSDB data
- **Fallback:** If master index missing, expired, or `PHASE 0 STATUS` is `failed`:
  - Alert: "[!] Phase 0 cache not found or invalid. Run Phase 0 first OR execute live queries."
  - Execute `show isis database | display json | no-more` directly

**Procedure:**
1. Load master index from `results/cache/isis_data_collection_*.json` (find latest)
2. Read preamble - surface any temporal warnings before proceeding
3. Verify cache expiry and Phase 0 completion via JSON fields
4. Load LSDB data from `cache_files.isis_database`
5. Perform LSDB sync analysis
6. Generate report
```

The reader might now ask - how would you craft the above prompt yourself? Same answer as before: ask the LLM. The analysis phase snippet above was generated from something close to a single line:

I want to perform analysis of the ISIS LSDB. Please create a prompt that checks data for consistency via master-index, checks if data is expired, and if data is missing or expired performs a live query via JMCP.

One sentence covers the intent - the LLM will typically fill in the two-gate validation, the fallback logic, the preamble read order, and the file path pattern. As with the Phase 0 helper prompt, the gap between this one-liner and the full snippet above is closed by the LLM's understanding of what a well-structured analysis phase should look like.

1.10 Token Savings

The empirical numbers were covered in the Introduction: 26.9 MB of raw output, nearly 10 million tokens - a payload large enough to put severe pressure on, or outright exhaust, most models' context windows. With

enhanced JMCP artifact mode only ~200 tokens reached the LLM; with standard JMCP under VS Code the context survived only because of VS Code's client-side guardrail. A raw API client would have received all 26.9 MB inline.

But the Phase 0 pattern is not primarily about surviving large payloads - it is about collecting data that can actually be compared.

The author has troubleshooting and data capture in mind. For that task, what matters is that all commands are executed within the same time window - not 10 or 20 minutes apart. An LSDB sync check, for example, is unreliable if router snapshots were collected an hour apart: the LSDB may have changed in between.

A further advantage of a single collection phase is cross-correlation. Assume analysis of the LSDB reveals missing LSPs or mismatching sequence numbers. Further investigation would be required - for example, checking queue statistics, interface errors, DDOS protection counters, CPU load, ISIS queue stats, and LSP statistics. Ideally, all of that information is collected in a single pass. This allows issues to be correlated using data from the same point in time - or, better still, with two snapshots taken in close succession to enable trending. Effective correlation requires contemporaneous data, not an attempt to diagnose an issue with outdated snapshots.

Phase 0 collects everything first - in parallel where possible, sequential per-command to keep the context window clean - and saves it to files. Only then does analysis begin, in a fresh session with a reset token budget.

The session boundary is the mechanism. A monolithic workflow - collect, parse, analyse, collect again, repeat - accumulates tokens continuously in a single session. Phase 0 breaks that chain: by the time analysis starts, the collection session has ended and its tokens are gone. Each analysis phase loads only the data subset it needs from the cache files - a targeted extract of a few kilobytes, not the full dataset.

```
Phase 0 (Collect): all commands -> all routers (parallel) -> files -> END SESSION
                    v one consistent time window
Phase 1a (Analyse): load isis_database cache -> LSDB checks -> report -> END SESSION [parallel]
Phase 1b (Analyse): load isis_adjacency cache -> adj. checks -> report -> END SESSION [parallel]
Phase 1c (Analyse): load isis_statistics cache -> perf. checks -> report -> END SESSION [
    parallel]
                    v all Phase 1 sub-tasks complete
Phase 2 (Correlate): load Phase 1 reports -> causal chains -> summary -> END SESSION
Phase 3 (Trend): load Phase 0 + Phase 2 outputs (multiple runs) -> trend report
```

The full 26.9 MB never needs to exist in any single LLM session. Each phase gets a clean slate and a focused slice of the data.

Environment warning: How well this holds in practice depends heavily on the LLM client and model being used. Some clients will silently compact or truncate the conversation when context pressure builds - if you see a "compacting conversation" notice, or responses start losing awareness of earlier steps, the context window is under pressure. To baseline your specific environment before running a large workload, use the canary rule test described in Part 2: a deliberately triggered rule violation confirms the model is still reading the full context. If the canary fails post-run, the session has been compromised and results should not be trusted.

Fresh window vs. compacted session - when each is appropriate:

When compaction kicks in, the instinct is often to continue in the same session - the model still responds, and the summary seems to cover the key context. This is usually the wrong call for workflow automation. The LLM writes its own compaction summary, and that summary is lossy: specific file paths, partially completed steps,

router counts, and error states may be paraphrased or silently dropped. The model appears to remember context, but the details it acts on can be subtly wrong.

Prefer a fresh session when:

- The current phase has written all its outputs to disk and the session's only remaining purpose is to hand off to the next phase - in that case, the file is the persistent state, and the conversation history adds nothing
- The canary rule fails or you observe the model losing awareness of earlier steps
- You are about to start a new analysis phase - the Phase 0 / phase boundary pattern already requires this; starting fresh is the correct behaviour, not a workaround

Keep the compacted session only when:

- There is an active reasoning chain that has not yet been written to any file - for example, a partial analysis where intermediate conclusions are still in context but no report has been saved yet. In that case, write a brief handover note to a file first (a *session_state*{epoch_timestamp}.txt under results/ works well), then start fresh from that file.

The general rule: once a phase's outputs are saved to disk, the conversation that produced them is scaffolding. Discard it and start the next phase with a clean context window and accurate context, rather than a compacted window where an unknown fraction of the budget has already been consumed by a summary you cannot verify.

1.11 File Naming Conventions

File naming might seem like a detail, but in an LLM-driven multi-phase workflow it is load-bearing infrastructure. The LLM constructs file paths from context file instructions, resolves the latest cache file using glob patterns, and the Master Index stores those paths as literal strings. A poor naming scheme undermines all three.

1.12 Common approaches - and why they fall short

The most natural instinct is a flat, descriptive name:

```
isis_database.json
isis_adjacency.json
```

This works fine for a one-off script, but fails immediately when Phase 0 runs twice: the second run silently overwrites the first. There is no way to distinguish a fresh capture from a stale one without opening the file.

The next instinct is a human-readable date prefix:

```
2025-02-21_isis_database.json
2025-02-21_isis_adjacency.json
```

Better - multiple runs no longer collide. But this format has a subtle problem in automation: the LLM (or a script) trying to find the most recent file has to parse a date string, handle timezone ambiguity, and deal with locale-dependent separators. It is also not sortable lexicographically unless the format is strictly ISO 8601 - a convention that is easy to violate by accident.

1.13 Why Unix epoch wins for cache files

A Unix epoch timestamp - seconds since 1970-01-01T00:00:00Z - is a plain integer:

- **Lexicographically sortable**: 1733558400 > 1733472000 without any parsing
- **Unambiguous**: no timezones, no locale, no separators to vary
- **Glob-friendly**: isis_database_*.json finds all versions; sort | tail -1 gives the latest
- **LLM-friendly**: the model can compare two epoch values with a single greater-than check - no date library needed

The recommended naming pattern is {command}_{epoch_timestamp}.json:

```
results/cache/isis_database_1733472000.json
results/cache/isis_adjacency_1733472000.json
results/cache/isis_interface_1733472000.json
```

This is already in use - the Master Index Structure section earlier in this blog shows exactly this pattern in the cache_files JSON block, and the Phase 0 Example uses {epoch_timestamp} as a placeholder for the same epoch value.

All files from a single Phase 0 run share the same epoch - the timestamp at which the collection started. This means the Master Index file path itself encodes the run:

```
results/cache/isis_data_collection_1733472000.json <- master index for this run
```

Finding the latest run is then a single glob + sort, which is exactly what the analysis phase context file instructs the LLM to do.

1.14 Directory structure - phase as a folder, not a filename prefix

A second common mistake is encoding the phase into the filename:

```
phase0_isis_database_1733472000.json <- brittle, hard to glob
phase1_lsdb_report_1733472000.json
```

The better approach is to use the directory as the phase boundary. Files naturally fall into their phase's folder, and a glob within that folder always targets the right scope:

```
results/
  cache/ <- Phase 0: raw cache files + Master Index
  phase1/ <- Phase 1: domain analysis reports (one file per sub-task)
  phase2/ <- Phase 2: executive summary and causal chain findings
  phase3/ <- Phase 3: trend reports (spans multiple Phase 0 runs)
```

Full example:

```
results/cache/isis_database_1733472000.json
results/cache/isis_adjacency_1733472000.json
results/cache/isis_data_collection_1733472000.json <- master index

results/phase1/lsdb_report_1733472000.json
results/phase1/adjacency_report_1733472000.json
results/phase1/performance_report_1733472000.json

results/phase2/isis_health_executive_1733472000.json

results/phase3/isis_trend_1733472000.json <- spans multiple runs (optional)
```

To get the LLM to follow this layout from the start, add one line to your Phase 0 context file:

store Phase 0 cache files under results/cache/, Phase 1 domain reports under results/phase1/, Phase 2 summary under results/phase2/, and trend reports under results/phase3/ - use unix epoch suffix on all filenames

1.15 Interface Transformations

1.16 The Problem

A common task in network automation is transforming logical interface names to physical interface names (and vice versa). A typical use case is collecting interface statistics at the IFD (physical interface) level for all interfaces running ISIS. show isis interface returns the IFLs (logical interfaces) configured for ISIS - for example, ge-0/0/0.0. To retrieve statistics for the parent physical interface, show interfaces extensive ge-0/0/0 must be used, without the trailing .0.

1.17 Solution: Pre-compute and Cache

Instead of transforming on-the-fly in every analysis step, do it once in Phase 0 and cache the mapping.

Phase 0 Enhancement:

```
# Phase 0: ISIS Data Collection (Enhanced)

**Procedure:**
...
4. **Build Interface Mappings** (do transformation ONCE):
   - Load `isis_interface_{epoch_timestamp}.json` from cache
   - Extract logical interfaces for each router (ge-0/0/2.0, ge-0/0/3.0, etc.)
   - Transform logical -> physical (ge-0/0/2.0 -> ge-0/0/2)
   - Exclude loopback interfaces (lo0.0)
   - **Save to file**: `results/cache/interface_mappings_{epoch_timestamp}.json`

**Interface Mapping File Structure:**
```json
{
  "timestamp": "2025-02-21T14:30:00Z",
  "routers": {
    "R11_re": {
      "logical_interfaces": ["ge-0/0/2.0", "ge-0/0/3.0", "ge-0/0/4.0"],
      "physical_interfaces": ["ge-0/0/2", "ge-0/0/3", "ge-0/0/4"]
    },
    "R12_re": {
      "logical_interfaces": ["ge-0/0/1.0", "ge-0/0/5.0"],
      "physical_interfaces": ["ge-0/0/1", "ge-0/0/5"]
    }
  }
}
```

As with the other patterns in this series, the LLM can generate this step for you. A one-liner is enough to get started:

create a prompt for a context file that loads the latest show isis interface output and builds a mapping file per router translating logical interfaces (e.g. ge-0/0/0.0) to their physical counterparts (e.g. ge-0/0/0). include a unix epoch timestamp in the filename

The Phase 0 snippet above collected the interface mapping and saved it to file. The following shows how a Phase 1 context file can load that cached data for analysis, without re-querying devices.

Reuse in Analysis Phases:
```

```
# Phase 1c: Interface Error Analysis

**Data Source:** `results/cache/interface_mappings_*.json` (from Phase 0 - find latest)

**Procedure:**
1. Load interface mappings file (latest by glob + sort)
2. Load `interfaces_extensive` cache from the path in the master index (`cache_files.
    interfaces_extensive`)
3. For each router, filter entries by the `physical_interfaces` list from the mappings file
4. Analyze statistics (errors, CRC, drops)

**Fallback:** If mappings file missing, or master index `cache_files.interfaces_extensive` is
    absent or expired, fall back to live queries:
   - Execute `show isis interface | display json | no-more`
   - Transform logical -> physical (strip trailing unit suffix, e.g. `ge-0/0/0.0` -> `ge-0/0/0`),
       exclude `lo0`
   - Save to `results/cache/interface_mappings_{epoch_timestamp}.json`
   - Proceed with analysis
```

1.18 Split Workflows for Clarity and Reliability

Large workflows should be split into multiple context files, each with a focused scope. The motivation is not primarily about hitting a token limit - modern LLM clients handle large tool outputs more gracefully than the raw numbers suggest (see Token Savings above). The real reasons are scope isolation, reliability, and parallelism.

A monolithic context file that runs data collection, multi-step analysis, and summary generation in a single session mixes three distinct tasks. Each step carries the noise of all previous steps - partial results, intermediate tool calls, accumulated reasoning - into the next. When something goes wrong, the entire run has to be repeated. When collection is slow, analysis is blocked.

Splitting by phase solves all three problems.

1.19 Split Into Phases

```
+ - - - - - - - - - - - - - - - - - - - - - - - - - - - +
| Phase 0: Collect (isis_collect.md) |
| Query all devices -> save to cache -> write Master Index |
| END SESSION |
+ - - - - - - - - - - - - - - - - - - - - - - - - - - - +
                    v one consistent time window
+ - - - - - - --+ - - - - - - - - - + - - - - - - - --+
| Phase 1a: Analyse | Phase 1b: Analyse | Phase 1c: Analyse | <- all in parallel
| LSDB (isis_lsdb.md)| Adj & Iface (adj.md) | Perf (isis_perf.md)|
| -> lsdb_report.json | -> adjacency_report.json| -> perf_report.json |
+ - - - - - - --+ - - - - - - - - - + - - - - - - - --+
                    v all Phase 1 sub-tasks complete
+ - - - - - - - - - - - - - - - - - - - - - - - - - - --+
| Phase 2: Correlate (isis_correlate.md) |
| Load all Phase 1 reports -> identify causal chains -> summary |
| END SESSION |
+ - - - - - - - - - - - - - - - - - - - - - - - - - - --+
                    v (optional - requires multiple Phase 0 runs)
+ - - - - - - - - - - - - - - - - - - - - - - - - - - --+
| Phase 3: Trend (isis_trend.md) |
| Compare snapshots over time -> detect drift -> trend report |
+ - - - - - - - - - - - - - - - - - - - - - - - - - - --+
```

Each phase starts with only the data it needs - loaded from the cache files written by Phase 0. The context is clean, the scope is bounded, and the session ends as soon as the phase output is saved to disk.

How parallelism works in practice: A context file is a passive instruction document - it cannot spawn or launch another session by itself. The parallel execution of Phase 1 sub-tasks shown above requires an

external trigger: either a human opening multiple chat windows simultaneously and starting each sub-task manually, or an external orchestrator (a shell script, CI pipeline, or multi-agent framework such as LangGraph or AutoGen) that launches each phase as a separate agent invocation in parallel. The phase/file architecture works identically in all cases - because the handoff between phases is always a file on disk, not an in-memory state. Multi-agent orchestration is a valid and increasingly common approach for automating this kind of workflow, but it is out of scope for this blog series.

1.20 Key Benefits

- 1. **Scope isolation**: Each phase and sub-task has a single, bounded task. The LLM starts clean - no collection noise, no partial tool results from earlier unrelated steps
- 2. **Parallel execution**: Phase 1 sub-tasks (1a, 1b, 1c) can run simultaneously after Phase 0 completes - collection is the only sequential dependency
- 3. **Failure isolation**: If Phase 1b fails, Phases 1a and 1c results are preserved and only Phase 1b needs to be re-run
- 4. **Data consistency**: All commands are collected in a single Phase 0 time window - all Analyse sub-tasks work from the same internally consistent snapshot (see Master Index Pattern)
- 5. **Incremental development**: Each phase and sub-task can be built and tested independently before assembling the full workflow

1.21 Data Sharing Between Phases

Each phase reads from the cache files written by Phase 0 and writes its own output to a phase-specific directory. The handoff mechanism is always a file on disk - no in-memory state passes between sessions.

```
Phase 0 -> Stores: results/cache/isis_data_collection_{epoch_timestamp}.json <- master index
                   results/cache/isis_database_{epoch_timestamp}.json
                   results/cache/isis_adjacency_{epoch_timestamp}.json
                   ...
                   v
Phase 1a -> Reads: master index + isis_database cache
            Writes: results/phase1/lsdb_report_{epoch_timestamp}.json

Phase 1b -> Reads: master index + isis_adjacency + isis_interface + interfaces_extensive cache
            Writes: results/phase1/adjacency_report_{epoch_timestamp}.json

Phase 1c -> Reads: master index + isis_statistics + ddos_protection + system_resources cache
            Writes: results/phase1/performance_report_{epoch_timestamp}.json
                   v all three complete
Phase 2 -> Reads: results/phase1/lsdb_report_*.json
                   results/phase1/adjacency_report_*.json
                   results/phase1/performance_report_*.json
            Writes: results/phase2/isis_health_executive_{epoch_timestamp}.json
                   v (optional - requires multiple Phase 0 runs)
Phase 3 -> Reads: results/cache/isis_data_collection_*.json (multiple epochs)
                   results/phase2/isis_health_executive_*.json (multiple epochs)
            Writes: results/phase3/isis_trend_{epoch_timestamp}.json
```

Phase report structure - what each analysis phase writes

For Phase 2 to work, each Phase 1 report must be structured so it can be loaded, scanned, and summarised efficiently. The same preamble + JSON pattern used for the Master Index applies here: a plain-text status header followed by a structured findings block.

```
PHASE 1a STATUS : complete | 74/74 routers analysed | 3 findings
ANALYSIS : LSDB sync check, LSP fragmentation, LSP lifetime
CRITICAL : R42 - stale LSP (seq 0x00003f21, neighbours expect 0x00003f24)
```

```
WARNINGS : 2 routers with LSP fragment count > 10
CLEAN : 71/74 routers

 -

```json
{
  "phase": "1a",
  "domain": "lsdb",
  "findings": [
    {
      "router": "R42",
      "severity": "critical",
      "type": "stale_lsp",
      "detail": "LSP seq 0x00003f21, neighbours expect 0x00003f24"
    },
    {
      "router": "R17",
      "severity": "warning",
      "type": "lsp_fragment_count",
      "detail": "fragment count 14, threshold 10"
    }
  ]
}
```

The plain-text preamble means Phase 2 can read the critical findings from all Phase 1 reports without parsing full JSON first. The router field in each finding entry is the join key for cross-correlation - Phase 2 builds a per-router index by collecting all findings where the same router name appears across multiple Phase 1 sub-task reports. The per-router finding list in the JSON block is what enables cross-correlation.

As before, you don't need to design this report structure from scratch. The following natural-language prompt serves as a good start:

I have collected ISIS device data in Phase 0 and stored it to results/cache/. Now create a Phase 1a analysis context file that loads the isis_database cache via the master index, performs an LSDB sync check and LSP fragmentation analysis across all 74 routers, and writes a report to results/phase1/lsdb_report_{epoch_timestamp}.json using a plain-text preamble followed by a JSON findings block. The preamble should summarise router counts, critical findings, and warnings on a single line each. The JSON block should list each finding with router name, severity, type, and detail fields.

The prompt above targets Phase 1a: it names the isis_database cache file and asks for LSDB-specific checks. To generate a Phase 1b or 1c context file, use the same prompt but replace those two details - the cache file (e.g. isis_adjacency for Phase 1b, isis_statistics for Phase 1c) and the analysis type (adjacency stability checks, performance checks). Everything else stays identical: preamble + JSON output format, results/phase1/ as the save location, and the same router / severity / type / detail field names in the JSON findings block.

That consistency is not accidental - it is what makes Phase 2 work. When all Phase 1 sub-tasks produce reports in the same structure and directory, Phase 2 can load them with a single glob pattern and join their findings on the router field without any format negotiation.

> **Note**: In a mature workflow, the shared report schema - field names, preamble layout, output directory - would be defined once in a Master Runbook and referenced by each phase context file, rather than repeated in full across 1a, 1b, and 1c. The Master Runbook pattern, including how to structure it as a central entry point and how individual phase context files reference it, is covered in Part 6 of this blog series.
```

1.22 Phase 2: Correlate

The real value of Phase 2 is not summarisation - it is correlation. Individual Phase 1 reports tell you what is wrong on each router. Phase 2 tells you why, by looking across domains simultaneously.

Why single-domain analysis misses the root cause

Consider a realistic scenario across the 74-router lab:

- **Phase 1a (LSDB)** reports: R42 has a stale LSP - its sequence number is behind what its neighbours expect. Flagged as an anomaly, cause unknown.
- **Phase 1b (Adjacency & Interface)** reports: R42 has 3 adjacency flaps on ge-0/0/2 within the collection window. Reported as an instability event, no LSDB context.
- **Phase 1c (Performance)** reports: R42 shows sustained COS queue drops on ge-0/0/2 (from show interfaces extensive). Separately, DDOS protection counters show elevated aggregate IS-IS PDU receive rates at the Routing Engine - consistent with the burst of Hello and LSP traffic generated by repeated adjacency flap reconvergence on that interface.

None of these sub-tasks can produce the full picture alone. Phase 1a sees the symptom in the control plane. Phase 1c sees the cause in the data plane. Phase 1b sees the intermediate effect. Only by reading all three together does the chain become clear:

Root cause chain: COS queue congestion on ge-0/0/2 -> IS-IS PDU drops -> adjacency flaps -> LSDB not refreshed -> stale LSP observed by neighbours.

A single-session monolithic workflow would have all of this data available, but the LLM would be reasoning across 10 different analysis tasks simultaneously, with tool call results from 74 routers still in context. The causal chain is far more likely to surface clearly in a Phase 2 session that starts clean, loads three compact structured reports, and is given a single focused instruction: find routers that appear in more than one report and explain the relationship.

Phase 2 context file - the cross-correlation instruction

```
## Phase 2: Correlate and Executive Summary

**Data Source:**
- `results/phase1/lsdb_report_*.json` (find latest)
- `results/phase1/adjacency_report_*.json` (find latest)
- `results/phase1/performance_report_*.json` (find latest)

**Procedure:**
1. Load all three Phase 1 reports. Read each preamble first - note any CRITICAL findings
2. Build a per-router index: for each router that appears in any finding, record which sub-tasks flagged it and what the finding was
3. **Cross-correlate**: identify routers flagged in more than one sub-task. For each, determine whether the findings share a plausible causal relationship (e.g. interface errors -> adjacency flaps -> LSDB staleness)
4. Produce the executive summary with three sections:
   - **Confirmed root causes**: routers where a cross-domain causal chain was established
   - **Isolated findings**: routers flagged in only one sub-task, no corroborating evidence
   - **Clean**: routers with no findings across all sub-tasks
5. Save to `results/phase2/isis_health_executive_{epoch_timestamp}.json`
```

As with all other patterns in this series, you do not need to write this from scratch. The one-liner below produces a working draft that you can then refine:

create a Phase 2 correlate context file that loads all Phase 1 domain reports from results/phase1/, cross-correlates findings per router across all sub-tasks, identifies causal chains, and generates an executive

summary split into confirmed root causes, isolated findings, and clean routers. Save to results/phase2/ using the same preamble + JSON structure and Unix epoch filename convention established in Phase 0.

1.23 Phase 3: Trend

A single Phase 2 report tells you what is wrong today. Phase 3 answers a different question: **is the network getting better, worse, or oscillating?**

Trending requires at least two complete runs - two Phase 0 collections at different times, each paired with its own Phase 2 executive summary. Phase 3 loads all available runs, aligns them by their collection epoch, and compares per-router findings across them. Its output is a trend report classifying each router into one of five states:

State	Meaning
Stable-clean	No findings in any run
Recurring	Flagged in two or more consecutive runs - the problem persists
First-occurrence	Flagged in the latest run only - new event
Improving	Flagged in earlier runs but clean in the latest
Oscillating	Alternates between clean and flagged - intermittent fault

Phase 3 reads from results/cache/ (Master Indexes - to read collection timestamps) and results/phase2/ (executive summaries - to read per-router findings). Its output goes to results/phase3/.

Unlike Phases 0–2, Phase 3 is optional and does not block the rest of a run. It can be triggered manually after a suspected regression, or scheduled automatically after each Phase 2 completes.

1.24 Generating a Trending Context File

As with the other phases, ask the LLM to generate the context file rather than writing it from scratch. The following one-liner is enough to produce a working draft:

I run a multi-phase ISIS health check workflow on a repeating schedule. Phase 0 saves device data to results/cache/ with Unix epoch timestamps in the filename - each run produces a master index matching results/cache/isis_data_collection_*.json. Phase 2 saves an executive summary for each run to results/phase2/isis_health_executive_*.json in preamble + JSON format with per-router findings. Create a Phase 3 trending context file that loads the 5 most recent Phase 0 master indexes and their paired Phase 2 summaries, aligns them by collection epoch, compares per-router findings across runs to classify each router as stable-clean, recurring, first-occurrence, improving, or oscillating, and saves a trend report to results/phase3/isis_trend_{epoch_timestamp}.json in the same preamble + JSON structure.

The prompt encodes the key decisions: look-back window (N=5, configurable), file glob patterns, the per-router classification scheme, and the output format. The LLM will fill in the comparison logic, the preamble status header, and the JSON schema for the trend report.

When to run Phase 3:

- After at least 2 full workflow runs have completed (Phase 0 -> Phase 1 -> Phase 2)
- As an automated post-Phase-2 step in a scheduler
- On demand after a suspected regression - comparing today's run against the last known clean state

Phase 3 trend analysis - including Redis TTL-based automatic expiry of old runs and configuring the look-back window - is covered further in Part 5.

1.25 Summary

In this part, we covered:

- **Phase 0 (Collect)**: separate data collection from analysis; query all devices once in a consistent time window; write a Master Index
- **Phase 1 (Analyse)**: parallel sub-tasks, each scoped to one data domain (1a: LSDB, 1b: Adjacency & Interface, 1c: Performance), each producing its own structured report
- **Phase 2 (Correlate)**: load all Phase 1 reports, identify cross-domain causal chains per router, write an executive summary
- **Phase 3 (Trend)**: compare Phase 0 snapshots and Phase 2 summaries across multiple runs to detect recurring failures, first-occurrence events, and improving or oscillating health
- Master Index pattern: preamble + JSON hybrid for temporal consistency and two-gate validation
- File naming conventions: Unix epoch timestamps for lexicographic sorting, glob-friendly discovery, and LLM-friendly comparison
- Interface transformation pre-computation: build IFL -> IFD mapping once in Phase 0, reuse across all Phase 1 sub-tasks
- Fresh-session discipline: why each phase starts in a clean context window and how to handle compaction

One final reminder before moving on: the single most useful thing you can do with any context file - whether it is a Phase 0 collection script, a Phase 1 analysis template, or a Phase 3 trending runbook - is run the **Contract Lint** prompt against it before first use. It systematically checks for ambiguous thresholds, undefined wait times, missing severity mappings, and logical gaps that cause inconsistent LLM behaviour at runtime. No amount of structural elegance compensates for an ambiguous instruction the LLM silently interprets the wrong way. The Contract Lint prompt and the full linting workflow are covered in Part 3.

In Part 5, we'll explore Redis as a high-performance caching layer, including installation, TTL management, performance comparisons between Redis and file-based storage, and how Redis enables Phase 3 trending with automatic expiry of old runs.

1.26 Next in the Series

Coming up in Part 5:

- Redis installation and configuration
- TTL (Time To Live) for automatic expiry
- Master Index with Redis integration

- Performance comparisons: Redis vs Files vs Direct queries
- When to use Redis vs file-based storage
- Scaling to large fleets

1.27 Glossary

- Artifact mode: An operating mode of the enhanced JMCP server in which command output is stored directly to Redis or files by the server itself - raw data never enters the LLM context window. The LLM receives only a brief summary (~200 tokens) regardless of output size.
- Cache file: A JSON file written by Phase 0 containing the collected output for a single command across all routers. Named with a Unix epoch suffix and stored under results/cache/. Analysis phases load only the cache files they need rather than re-querying devices.
- Canary rule: A deliberately incorrect rule or constraint placed in a context file to verify that the LLM is still reading the full context. If the canary is not triggered after a long session, the context window may have been silently compacted. Described in Part 2 of this series.
- Client-side guardrail: A protection applied by the LLM client (not the MCP protocol) to handle oversized tool results. VS Code Copilot chat, for example, offloads large tool responses to a temporary file rather than injecting them as raw text. This behaviour is not guaranteed by raw API clients.
- Context window: The maximum amount of text (measured in tokens) that an LLM can hold in a single session. Includes the system prompt, conversation history, tool call results, and the model's own responses.
- Cross-phase correlation: The process of comparing findings across multiple Phase 1 domain reports to identify causal relationships that no single sub-task could detect alone. Performed by Phase 2 (Correlate), which loads all Phase 1 reports and builds a per-router view across domains.
- cRPD: Containerised Routing Protocol Daemon - a Junos routing protocol stack packaged as a container. Used in this series as the lab environment (74 ISIS-meshed cRPD instances).
- IFD (Interface Device): The physical interface layer in Junos (e.g., ge-0/0/0). Contrast with IFL.
- IFL (Interface Logical): A logical sub-interface in Junos (e.g., ge-0/0/0.0). ISIS adjacencies and routing protocols are configured on IFLs; interface statistics are reported at the IFD level.
- Interface mapping: A pre-computed file that translates logical interface names (IFLs) to physical interface names (IFDs) for all routers. Built once in Phase 0 and reused by analysis phases that need to query show interfaces extensive.
- ISO 8601: International timestamp format (e.g., 2025-02-21T14:30:00Z) that is human-readable and sortable but more verbose than Unix epoch.
- JMCP: Junos MCP server - an MCP server that exposes Junos CLI commands as tools callable by an LLM. Available as a standard version (returns raw CLI output to the client) and an enhanced version with built-in Redis/file storage.
- LSDB (Link State Database): ISIS's distributed database of network topology. Each router maintains a copy; all copies should be identical (synchronised) across the network. Inconsistencies indicate a routing problem.
- LSP (Link State PDU): An ISIS protocol data unit that carries topology information. Each router originates its own LSP and floods it to neighbours. A stale LSP (sequence number behind what neighbours expect) indicates the originating router is not refreshing its topology information.
- Master Index: A metadata file generated at the end of Phase 0. Contains a plain-text preamble with collection status and temporal warnings, followed by a JSON block with cache file paths, per-router

collection timestamps, and execution summary. Used by all analysis phases to locate their input data.

- MCP (Model Context Protocol): An open protocol that allows LLMs to call external tools (e.g., network device queries, file operations). Tools are exposed by an MCP server; the LLM client decides when to invoke them.
- Monolithic workflow: A single-session approach where all data collection and analysis happens sequentially in one LLM context. Mixes concerns, prevents parallelism, and means a failure in any step requires restarting from scratch.
- Phase 0: The data collection phase of a multi-phase workflow. Queries all required commands from all devices, saves each output to a cache file immediately, and generates a Master Index. All subsequent analysis phases read from Phase 0's cache rather than re-querying devices.
- Preamble: The plain-text header written at the top of a Master Index or phase report file, before any JSON. Contains status, timestamps, and warnings in human-readable form so that any LLM or human loading the file sees critical information immediately without parsing.
- Redis: An in-memory data store used by the enhanced JMCP server to persist collected data. Supports TTL (automatic expiry), fast key-based lookup, and cross-session access. Covered in depth in Part 5.
- Scope isolation: The principle that each phase in a split workflow has a single, bounded task. The LLM starts a phase session with only the data relevant to that phase, avoiding noise from previous tool calls, partial results, or unrelated reasoning.
- Temporal consistency: Ensuring that data being compared was collected within an acceptable time window. An LSDB sync check, for example, is only meaningful if all router snapshots were collected at approximately the same time. Enforced by the Master Index's temporal skew check.
- Temporal skew: The difference in collection time between the earliest and latest router in a Phase 0 run. Routers collected more than 60 seconds apart from the earliest are flagged in the Master Index preamble as potentially unreliable for sync analysis.
- tiktoken: OpenAI's token counting library. Used in this series to measure the token count of raw command output (e.g., 26.9 MB = 9,952,294 tokens for show isis database extensive across 74 routers).
- Token budget: The portion of the context window available for a session's work. Starts fresh at the beginning of each new session. Split workflows preserve the full budget for each phase by ending sessions at phase boundaries.
- Trend phase (Phase 3): The optional phase that compares Phase 0 snapshots and Phase 2 executive summaries across multiple workflow runs over time. Classifies each router as stable-clean, recurring, first-occurrence, improving, or oscillating. Requires at least two complete runs and writes to results/phase3/.
- TTL (Time To Live): An expiry duration applied to cached data. The Master Index records a cache expiry 7 days from collection; analysis phases check this value before using cached data and fall back to live queries if expired. Also used by Redis to automatically delete stale keys.
- Unix epoch: Timestamp format representing seconds since 1970-01-01T00:00:00Z (e.g., 1733472000). Compact, timezone-agnostic, lexicographically sortable, and directly comparable with integer arithmetic - the recommended format for cache file naming in this series.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



